SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA17**

COURSE OUTCOME COVERED

CO2: *Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)*

Assessment Tool Weightage: 40%

Duration: 1 Hour

QUESTIONS: 3 **Total Marks:30**

Annexure A

Dry Run Evaluation

Code of Conduct:

I Sanjana.G (192511020) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

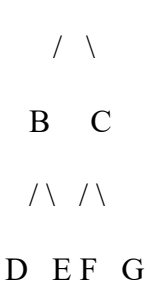
Sanjana.G

Annexure A

Dry Run Evaluation

1. Question 1 – Breadth-First Search (BFS)

Consider the following graph:



Task: Starting from node **A**, perform a **Breadth-First Search (BFS)**. Complete the following table.

Iteration	Queue	Visited Node
1		
2		
3		
4		
5		
6		
7		

Questions:

1. Write the BFS traversal order.
2. Show the queue contents after each iteration.

Given graph: A is the root, B and C are its children, D and E are the children of B, and F and G are the children of C.

1. Write the BFS traversal order.

Breadth-First Search (BFS) visits nodes level by level, from left to right, using a queue.

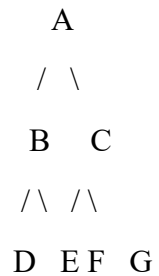
BFS Traversal Order: A → B → C → D → E → F → G

2. Show the queue contents after each iteration.

Iteration	Queue	Visited Node
1	B, C	A
2	C, D, E	B
3	D, E, F, G	C
4	E, F, G	D
5	F, G	E
6	G	F
7	Empty	G

Explanation: BFS removes the first node from the queue, visits it, and adds its unvisited children to the rear of the queue. The process continues until the queue becomes empty.

2. Depth-First Search (DFS) Using the same graph,



Iteration	Stack	Visited Node
1		
2		
3		
4		
5		
6		
7		

Perform **Depth-First Search (DFS)** starting from **A**.

1. Perform Depth-First Search (DFS) starting from A.

Depth-First Search (DFS) explores one branch as deeply as possible before backtracking. It uses a stack to keep track of nodes.

DFS Traversal Order: A → B → D → E → C → F → G

2. Complete the DFS table.

Iteration	Stack (Top at Left)	Visited Node
1	B, C	A
2	D, E, C	B
3	E, C	D
4	C	E
5	F, G	C
6	G	F
7	Empty	G

Explanation: DFS visits the leftmost unvisited node first and continues until it reaches a node with no unvisited children. It then backtracks and explores the remaining branches.

Complete the table.

3. **Initial State :**
   1 3 6
   5 0 2
   4 7 8

Goal State :
1 2 3
4 5 6
7 8 0

Task

Trace **Breadth-First Search (BFS)** until the goal state is reached.

Fill in the table.

Iteration	Current State	Queue Before Expansion	Queue After Expansion
1			
2			
3			
4			
5			

Questions

1. Which node is expanded in each iteration?
2. How many states are generated in total?
3. Write the complete solution path from the initial state to the goal state.
4. Why does BFS always find the shortest solution in an unweighted 8-puzzle problem?

1. Trace BFS until the goal state is reached.
BFS explores states level by level using a FIFO queue. The blank tile (0) can move up, down, left, or right whenever the move is valid.
For the table below, each state is written in compact form. For example, 136/502/478 represents three rows: 1 3 6, 5 0 2, and 4 7 8.

Iteration	Current State	Queue Before Expansion	Queue After Expansion
1	136/502/478	Initial state	106/532/478, 136/572/408, 136/052/478, 136/520/478
2	106/532/478	106/532/478, 136/572/408, 136/052/478, 136/520/478	136/572/408, 136/052/478, 136/520/478, 016/532/478, 160/532/478
3	136/572/408	136/572/408, 136/052/478, 136/520/478, 016/532/478, 160/532/478	136/052/478, 136/520/478, 016/532/478, 160/532/478, 136/572/048, 136/572/480
4	136/052/478	136/052/478, 136/520/478, 016/532/478, 160/532/478, 136/572/048, 136/572/480	136/520/478, 016/532/478, 160/532/478, 136/572/048, 136/572/480, 036/152/478, 136/452/078
5	136/520/478	136/520/478, 016/532/478, 160/532/478, 136/572/048, 136/572/480, 036/152/478, 136/452/078	016/532/478, 160/532/478, 136/572/048, 136/572/480, 036/152/478, 136/452/078, 130/526/478, 136/528/470

Note: These are the first five iterations only. BFS continues until the goal state is reached.
2. Which node is expanded in each iteration?
The node at the front of the queue is expanded in each iteration.

- Iteration 1: Initial state — 136/502/478
- Iteration 2: 106/532/478
- Iteration 3: 136/572/408
- Iteration 4: 136/052/478
- Iteration 5: 136/520/478

3. How many states are generated in total?
Using BFS with duplicate states excluded and the move order Up, Down, Left, Right, 500 unique states are discovered, including the initial state, before the goal is reached. The exact count can vary with the implementation's move order and duplicate-state handling.

4. Write the complete solution path from the initial state to the goal state.
The shortest solution requires 8 moves.
Step 1 – Initial State
1 3 6
5 0 2
4 7 8
Step 2 – Move Right
1 3 6
5 2 0
4 7 8
Step 3 – Move Up
1 3 0
5 2 6
4 7 8
Step 4 – Move Left
1 0 3
5 2 6
4 7 8
Step 5 – Move Down
1 2 3
5 0 6
4 7 8

Step 6 – Move Left
1 2 3
0 5 6
4 7 8
Step 7 – Move Down
1 2 3
4 5 6
0 7 8
Step 8 – Move Right
1 2 3
4 5 6
7 0 8
Step 9 – Move Right (Goal State)
1 2 3
4 5 6
7 8 0

5. Why does BFS always find the shortest solution in an unweighted 8-puzzle problem?
BFS explores all states at one depth before moving to the next depth. Since every valid move has the same cost, the first time BFS reaches the goal state, it has found a solution with the minimum number of moves. Therefore, BFS guarantees the shortest solution in an unweighted 8-puzzle problem.

RUBRICS FOR CALCULATION

Criteria	Weightage (%)	Level 4 – Exemplary (4)	Level 3 – Proficient (3)	Level 2 – Developing (2)	Level 1 – Beginning (1)
Understanding of Search Problem	**20**	Clearly understands the search problem, initial state, goal state, and search strategy.	Understands the problem with minor misunderstandings.	Demonstrates partial understanding of the search problem.	Shows little or no understanding of the search problem.
State Expansion and Dry Run Execution	**30**	Correctly traces every iteration, expands nodes accurately, and maintains the Queue/Stack/Priority Queue without errors.	Performs most iterations correctly with minor mistakes in state expansion or data structure updates.	Performs only some iterations correctly; several errors in tracing or node expansion.	Unable to correctly perform the dry run or expand states.
Application of Search Algorithm	**20**	Applies the selected algorithm (BFS/DFS/UCS) correctly, following all algorithmic rules.	Applies the algorithm correctly with a few procedural errors.	Applies only basic algorithm steps with noticeable mistakes.	Fails to apply the correct search algorithm.
Accuracy of Solution Path	**20**	Identifies the correct traversal order/solution path and goal state with proper justification.	Solution path is mostly correct with minor errors.	Partially correct solution path; goal may not be reached correctly.	Incorrect or incomplete solution path.
Presentation and Logical Reasoning	**10**	Queue/Stack/Priority Queue updates, state diagrams, and explanations are neat, organized, and logically presented.	Presentation is clear with minor organizational issues.	Limited explanation and organization; reasoning is partially clear.	Poor presentation with unclear or missing reasoning.